

Midterm

OperatorsActivity#2

Submitted by: Arevalo, Jay Clifferson P.

Course, Year & Section: BSIT 2-3

Course Code & Description: COMP 009 Object Oriented Programming

Submitted to: Sir SEVERINO M. BEDIS JR.

General Instructions

1. Create a Java program based on the questions.
2. Follow the exact required output shown in the question.
3. You must use the specific operator required in each number.
4. Do NOT hardcode the final answers (e.g., `System.out.println(86);` without computation is not allowed).
5. Show proper use of variables and correct operator usage.
6. Follow proper Java syntax, indentation, and naming conventions.
7. Use `System.out.println()` for displaying output.
8. Screenshot the code and the output.
9. Compile and run your program to verify correctness before submission.
10. Submission Format:
`Section_LastName_FirstName_OperatorsActivity#2.pdf`

Part I

Create three (3) separate Java programs following the required operators:

1. A program that uses bitwise operators.
 2. A program that uses the ternary operator (`? :`).
 3. A program that uses assignment operators (`+=`, `-=`, `*=`, `/=`).
- Each program must produce the exact output required in the question and must compute the result using the specified operator.

Part II

Create one Java program that demonstrates the use of multiple operators.

Requirements:

1. Use at least five (5) different operators from the following categories:
 - o Arithmetic operators
 - o Relational operators
 - o Logical operators
 - o Unary operators
 - o Assignment operators
 - o Bitwise operators
2. The program must produce exactly six (6) outputs only.
3. At least one output must contain a tricky increment or decrement expression, such as:
 - o `a++ + ++a`
 - o `--b + b++`
 - o `++x + x++`
4. Do not reuse variable names, values, or logic patterns from Part I.
5. The outputs must include:
 - o At least two (2) boolean results
 - o At least four (4) numeric results

PART 1 BITWISE

```

1 public class Bitwise {
2     public static void main(String[] args) {
3         int a = 6; // 0110
4         int b = 4; // 0100
5
6         System.out.println("a & b = " + (a & b));
7         System.out.println("a | b = " + (a | b));
8         System.out.println("a ^ b = " + (a ^ b));
9         System.out.println("~a = " + (~a));
10    }
11 }
12

```

Input/Output

Language version: JDK 21.0.0 ☒ Interactive Mode

Input arguments

Output Generated files

```

a & b = 4
a | b = 6
a ^ b = 2
~a = -7

```

Compiled and executed in 2.472 sec(s)

PART 1 TERNARY

The screenshot shows an IDE with a Java file named `TernaryExample.java`. The code is as follows:

```
1- public class TernaryExample {
2-     public static void main(String[] args) {
3-         int num = 67;
4-         String result = (num % 2 == 0) ? "Even MAN" : "Odd Folk innit";
5-         System.out.println("Number is " + result);
6-     }
7- }
```

The IDE's right-hand pane shows the output of the program:

```
Number is Odd Folk innit
```

Below the output, it states: "Compiled and executed in 3.069 sec(s)".

PART 1 ASSIGNMENT

The screenshot shows an IDE with a Java file named `AssignmentExample.java`. The code is as follows:

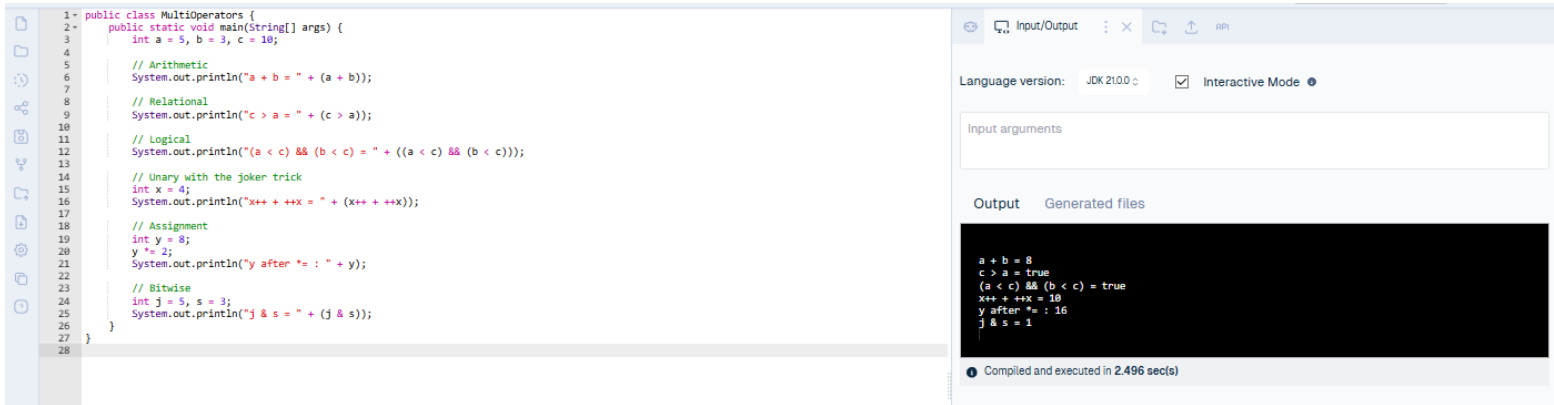
```
1- public class AssignmentExample {
2-     public static void main(String[] args) {
3-         int x = 20;
4-
5-         x += 5; // x = 25
6-         System.out.println("x after += : " + x);
7-
8-         x -= 10; // x = 15
9-         System.out.println("x after -= : " + x);
10-
11-         x *= 2; // x = 30
12-         System.out.println("x after *= : " + x);
13-
14-         x /= 3; // x = 10
15-         System.out.println("x after /= : " + x);
16-     }
17- }
```

The IDE's right-hand pane shows the output of the program:

```
x after += : 25
x after -= : 15
x after *= : 30
x after /= : 10
```

Below the output, it states: "Compiled and executed in 1.783 sec(s)".

PART 2 MULTIOPERATORS



The screenshot displays an IDE with a Java class named `MultiOperators` and its execution results.

Source Code:

```
1- public class MultiOperators {
2-     public static void main(String[] args) {
3-         int a = 5, b = 3, c = 10;
4-
5-         // Arithmetic
6-         System.out.println("a + b = " + (a + b));
7-
8-         // Relational
9-         System.out.println("c > a = " + (c > a));
10-
11-         // Logical
12-         System.out.println("(a < c) && (b < c) = " + ((a < c) && (b < c)));
13-
14-         // Unary with the joker trick
15-         int x = 4;
16-         System.out.println("x++ + ++x = " + (x++ + ++x));
17-
18-         // Assignment
19-         int y = 8;
20-         y *= 2;
21-         System.out.println("y after *= : " + y);
22-
23-         // Bitwise
24-         int j = 5, s = 3;
25-         System.out.println("j & s = " + (j & s));
26-     }
27- }
28
```

Execution Environment:

- Language version: JDK 21.0.0
- Interactive Mode: ☒

Input arguments:

Output:

```
a + b = 8
c > a = true
(a < c) && (b < c) = true
x++ + ++x = 10
y after *= : 16
j & s = 1
```

Generated files:

Compiled and executed in 2.496 sec(s)